

LLM-WIKI.md: Field Notes on a Knowledge Base That Maintains Itself

A Short List of Rules for Letting the Model Keep the Notes

Andrej Karpathy

Abstract. *This file exists because most personal knowledge systems die of maintenance, not of bad ideas. Collecting is easy, organizing is hard, and keeping fifty interlinked notes current is the work nobody does twice. The pattern here moves that work to the model: you curate sources and ask questions, the agent files, links, summarizes, and reconciles. The throughline is the same in every section: the human owns judgment and the raw record, the model owns the bookkeeping, and the wiki is a compiled artifact that compounds rather than a pile that grows.*

Index Terms. *LLM-maintained knowledge bases, Obsidian, Claude Code, markdown, compounding notes, retrieval versus compilation, second brain.*

I. SOURCES ARE IMMUTABLE

Everything you save lands in `raw/` and is never edited after it lands. Articles, transcripts, PDFs, screenshots: this is the source of truth, and its only job is to be the thing the wiki is built from. If a source is wrong, add a correcting source; do not rewrite history. The moment you start editing raw files by hand you have two systems of record and no way to tell which one is true.

II. SEPARATE THE LAYERS

Three layers, three owners. `raw/` holds immutable sources and belongs to you. `wiki/` holds generated pages and belongs to the model. A single schema file (`CLAUDE.md` or `AGENTS.md`) holds the rules and belongs to both. Do not blur them. When the model writes into `raw/`, or you hand-tune `wiki/` to win an argument, the boundaries that make the system trustworthy are gone.

III. THE MODEL OWNS THE WIKI

You rarely write a wiki page yourself. Your job is to choose what enters `raw/`, to ask questions, and to think. The model's job is the part humans avoid: summarizing, cross-referencing, filing under the right entity, and updating neighbors when something new arrives. If you find yourself doing the bookkeeping, the schema is underspecified, not the model.

IV. COMPILE, DON'T RETRIEVE

This is not RAG. RAG re-derives an answer from raw chunks on every query and accumulates nothing. Here the sources are compiled once into structured, linked pages, and questions are answered from that built artifact. The analogy holds: `raw/` is

source code, the model is the compiler, `wiki/` is the executable, queries are runtime. Knowledge that is compiled compounds; knowledge that is retrieved is rediscovered.

V. INGEST ONE SOURCE AT A TIME

Drop a single file into `raw/` and tell the model to ingest it. A good ingest is not one new page; it is the model tracing the implications of that source across the graph, touching every page the new fact changes. Batch-importing your entire digital life in a weekend produces a dump, not a wiki, because nothing gets linked while the pile is still forming.

VI. LINK EVERYTHING

Every page connects to others through wikilinks, and every wikilink is a visible edge in the graph. This is why Obsidian is the front-end of choice: the graph view shows clusters forming, hubs emerging, and orphans that nobody linked. An entity that appears in five pages but links to none is a sign the ingest was lazy. The value of the system is in the edges, not the nodes.

VII. NAVIGATE BY INDEX

The model should reach an answer by reading `index.md`, following the few relevant pages, and synthesizing, not by loading the whole vault into context. A wiki of a hundred articles and several hundred thousand words is still fast if the index is honest. If the model is brute-forcing the corpus on every question, the index has stopped reflecting the territory and needs a pass.

VIII. LINT THE KNOWLEDGE

Treat the wiki like code and run health checks. Ask the model to find contradictions between pages, surface low-confidence claims, list orphan pages, and flag entities that drifted into two spellings. A contradiction is information, not an error to paper over: it usually means two sources disagree and you now know where to look. Skipping the lint is how a wiki quietly rots while the graph still looks impressive.

IX. START SMALL

Begin with ten sources, not ten thousand. Get ingest, query, and lint to feel natural before you add a search engine, elaborate frontmatter, or a schema with twenty rules. The first few ingests need supervision; naming conventions will change and early pages will be messy, and that is normal. A small wiki you actually feed beats a beautiful architecture you abandon in week three.

AK